Kritika Sakhuja · Lot Plan & Lens.

ARRIVO · INNOTECH'26 · CATEGORY 6 · YOUR STUDY GUIDE

Your part: the map. That means:

- how the real KIET lot became a drawing you can tap
- the satellite photo it comes from
- how every bay was placed
- the two orientations (phone and laptop)
- the row zoom
- the **3D view** where buildings rise out of the plan
- the **X-ray lens** you hold and drag over the plan

This is the part judges remember, because it's the most visual.

The rest of the team:

- **Vaibhav:** the rest of the site and the booking flow
- **Prisha:** the server and database
- **Krish:** the problem and the impact

Your slides:

- **7:** Built on the real lot
- **8:** The live plan: the lens and 3D cards. Vaibhav adds one line about dark mode.
- **During the live demo (slide 10):** Vaibhav drives the mouse up to the booking. Then you take over: pick up the lens, drag it, switch to 3D, and talk while you do it.

How to use this guide. Read it twice. Then open `frontend/src/lot.jsx` (top half) and `frontend/src/LotMap.jsx` and find each piece of code named here. Practise the lens on a laptop *and* a phone until it feels natural.

1. Say this

> **Slide 7 · Built on the real lot.** "This isn't a made-up layout. We took a Google Earth capture of the lot beside the KIET football ground and marked the five rows in yellow. Then we used the photo's own scale bar: 50 metres is 385 pixels, so one metre is 7.7 pixels. Every bay on our map sits on the photo's own pixels. That gives 96 bays in five rows, and a standard bay comes out at 2.5 by 4.9 metres, which is a real car bay. That's why our map lines up with the real lot."

> **Slide 8 · The lens and 3D.** "There are three ways to read the lot. Tap any bay to open it. Pick up the X-ray lens, hold it and drag: every bay under it lights up as free, booked, ending soon or reserved, with the real satellite photo underneath and the bay's details in a card. Let go, and that bay opens. And the 3D view tilts the plan so the campus buildings rise out of it, so you recognise where you are."

> **Live demo.** "Let me pick up the lens. Yellow means free, black means a car is in it, a yellow outline means it's leaving soon, and a dashed white one means someone is on the way. The card says this bay is free till 16:30, and it counts what's inside the lens. I let go, and it opens that bay. Now 3D: the main block, the north wing and the south block rise out of the plan. It's all done in the browser with CSS, no 3D library."

2. From satellite photo to map

The photo

- **File:** `frontend/public/kiet-lot.jpg`, a **Google Earth** capture of the KIET main lot, Ghaziabad.
- **Size: 784 × 1601 pixels.**
- **Our marks:** the five rows of bays were marked on it in yellow.
- **Scale:** the capture has a scale bar where **50 m = 385 px**. So **1 metre = 7.7 pixels** (`PX_PER_M = 7.7`, `lot.jsx` line 11).
- **Credit:** the footer and slide 7 both say "Imagery © Google".

One coordinate system for everything

Every position in the map is a **pixel of that photo**:

- bays
- buildings
- trees
- the road
- the football pitch

That's the key idea. Because the drawing and the photo share coordinates, the drawing lines up with the photo exactly. That's how the lens can show the real photo under the bays.

```
// lot.jsx lines 10-12
export const IMG = { w: 784, h: 1601, rotate: -1, cx: 400, cy: 700 }
export const PX_PER_M = 7.7
export const VIEW = { x: 30, y: 226, w: 680, h: 930 }
```

VALUE	MEANING
`IMG.w, IMG.h`	The photo's size in pixels.
`rotate: -1, cx: 400, cy: 700`	The photo is turned **1 degree** around the point (400, 700) so the rows run perfectly straight up and down. Satellite captures are rarely perfectly aligned.
`VIEW`	The part of the photo we show: from (30, 226), 680 wide and 930 tall. That's just the lot, not the whole capture.

How the rows are described

Each row is described once, in `ROWS` (`lot.jsx` lines 14–75). Here is row A:

```
{ id: 'A', name: 'Ground Edge', axis: 'v', line: 547, from: 270, to: 910, count: 33, depth: 38, dir: -1 }
```

FIELD	MEANING	FOR ROW A
`axis`	`'v'` if the row runs up–down, `'h'` if it runs left–right.	up–down
`line`	The edge line the bays sit against (the kerb), as an x or y pixel.	x = 547
`from`, `to`	Where the row starts and ends along that line.	y = 270 to 910 (640 px = 83 m)
`count`	How many bays fit.	33
`depth`	How deep a bay is, in pixels.	38 px = **4.9 m**
`dir`	Which side of the line the bays are on (−1 = left/up, 1 = right/down).	left of the line

How 96 bays are made from 5 rows

The code (`SLOTS`, `lot.jsx` lines 93–126) splits each row evenly:

1. **Pitch** (one bay's share of the row) = (to − from) ÷ count. For row A, 640 ÷ 33 = **19.4 px = 2.5 m**.
2. Each bay gets a **2.6 px gap** from its neighbours (the painted line).
3. Each bay gets:
   - **an ID:** row letter + two-digit number, e.g. `A-14`
   - **its rectangle:** x, y, width and height in photo pixels, plus its centre
   - **a type:** standard, compact, EV or step-free
   - **its size in metres:** pixels ÷ 7.7, e.g. "2.5 × 4.9 m"

Be precise about this if asked. We traced each **row** by hand (its kerb line, start and end) and counted its bays. The code then spaces the bays evenly, because bays in one row are all the same width. We didn't draw 96 separate rectangles by hand.

Where the bay counts come from. Each row's measured length divided by a standard ~2.5 m bay:

ROW	LENGTH	÷ 2.5 M	BAYS
A	83 m	33.2	**33**

ROW	LENGTH	÷ 2.5 M	BAYS
B	24.7 m	9.9	**10**
C	60 m	24	**24**
D	16.4 m	6.5	**6**
E	58.7 m	23.5	**23**

If the painted bays on the ground turn out different, change `count` in **two places**: `ROWS` in `frontend/src/lot.jsx` and `ROWS` in `backend/server.py`. The server and the site must agree.

Before 30 October: walk past row D (only 6 bays) and count the painted bays. Then you can say "we checked on the ground".

The rows

ROW	NAME	BAYS	NOTES
A	Ground Edge	33	Nose-in bays along the football ground, off the service road. A-33 is step-free.
B	North Wing	10	Short shaded strip under the trees. All compact (₹15/h).
C	Main Block	24	Facing the main building. C-01 to C-04 have EV charging (₹35/h).
D	Corner	6	By the south-east entrance. D-01 and D-02 are step-free.
E	South Lot	23	The only row that runs left–right, beside the two-wheeler stand.

The types are set in `typeFor` (`lot.jsx` lines 84–89).

Everything around the bays

Traced on the same pixels, so you recognise the place:

- **Buildings** (`BUILDINGS`, lines 141–146). Each has a height `z` used in 3D:
  - the North wing (an odd-shaped polygon)
  - the Main block
  - the South-east block
  - the South block
- **Trees** (`TREES`, lines 148–166): 17 circles.
- **Roads, the football pitch, the two-wheeler stand** (marked "not bookable"): drawn in `ContextShapes` (`LotMap.jsx` lines 158–205).
- **Labels:** SERVICE ROAD, CAMPUS ROAD, KIET FOOTBALL GROUND, MAIN BLOCK (lines 208–230).

3. How the plan is drawn

The plan is one **SVG** (`PlanSvg`, `LotMap.jsx` lines 355–415).

Why SVG. It's made of shapes, not pixels, so:

- it stays sharp at any zoom
- every bay is a real element you can tap
- CSS can colour it (which is how dark mode and status colours work)

The SVG's `viewBox` is set to `VIEW`, in photo pixels. The browser scales it to fit any screen automatically.

One bay (`Bay`, lines 77–95)

Each bay is a group with:

- a rounded **box**, coloured by status
- a **hatch pattern** (diagonal stripes) if it's reserved
- a **car** if someone is parked. Cars come in 6 colours chosen by the booking, so the lot looks real. The car **drives in** from the road side and **drives out** when the booking ends, over 1 second (`Car`, lines 30–61).
- a **yellow ring** if the car leaves within 20 minutes
- a small **icon** for EV (lightning bolt) or step-free bays when they're empty

Two orientations (`ORIENT`, `lot.jsx` lines 171–186)

The lot is tall and narrow. That fits a phone held upright, but not a wide laptop screen.

- **Portrait** (screens up to 980 px wide): the photo as it is.
- **Landscape** (wider than 980 px): the whole drawing is **rotated −90°**, so north points left and the lot fills a wide screen.

The trick: shapes are drawn inside one group that gets rotated. But **text sits outside that group**, so labels and row letters always read upright. The north arrow turns to match. `SCREEN` (lines 194–205) works out where every bay ends up on screen in each orientation.

Animations on the plan

WHAT YOU SEE	HOW
Bays pop in, rippling out **from the gate**	Each bay's delay = its distance from the gate at (562, 1112) (`DELAY`, lines 15–18). CSS `bay-in` scales it from 30% to full. It starts when 18% of the map is on screen (an `IntersectionObserver`).
Cars drive in and out	Motion's `AnimatePresence`: the car slides 36 px from the road side and fades.
Selected bay	A design-tool selection (`SelectionShapes`, lines 267–328): yellow guide lines to the edges, a black box that shrinks onto the bay, four white corner handles. Plus a label like `A-14 · 2.5 × 4.9 m`.
Hover tooltip (mouse)	Bay ID, status and a short line like "free till 16:30" (lines 788–848).
Row zoom (All, A–E buttons)	Works out the row's box with 34 px padding, then scales (max 3.4×) and moves it to the centre. CSS animates the move over 1.1 s (lines 587–594).

WHAT YOU SEE	HOW
"Free right now" counter	A big rolling number on the ground beside the pitch, with a **20 m scale bar**: 20 × 7.7 px × screen scale.

4. The 3D view

No 3D library. It's done entirely with **CSS 3D transforms**: no Three.js, no WebGL. That keeps the site light, and it works on any phone.

Step 1 · Tilt the plan

```
/* islands.css 454-475 */
.map-persp { perspective: 2400px; perspective-origin: 50% 30%; }
.map-stage.is-tilt .map-tilt {
  transform: translateY(7%) rotateX(54deg) rotateZ(-32deg) scale(0.96);
}
```

- `perspective` makes far things smaller, like a camera.
- `rotateX(54deg)` tips the plan back like a table.
- `rotateZ(-32deg)` turns it for a corner view.
- It animates over **1.2 seconds**.

Step 2 · Raise the buildings

For every building (`Extrusions`, `LotMap.jsx` lines 431–473):

1. **Walls.** Take the building's outline. For each edge, make a flat `div` as long as that edge, turn it to the edge's angle (`rotateZ`), and stand it up (`rotateX(90deg)`). Its height = the building's `z`.
2. **Roof.** A `div` shaped like the outline (using `clip-path: polygon(...)`), lifted up by `z` (`translateZ`).
3. **Fake sunlight.** Each wall is shaded by the angle it faces, as if light comes from one side. That's why walls look lighter or darker.

Step 3 · Animate the rise

Wall height and roof lift are both multiplied by a CSS variable `--lift`:

- `0` = flat
- `1` = full height

Normally the browser can't animate a CSS variable. We **register** it as a number with `@property` (`islands.css` lines 1–5), so it can:

```
@property --lift { syntax: '<number>'; inherits: true; initial-value: 0; }
```

So the plan tilts first, then **0.35 s later** the buildings grow out of the ground over 1.1 s.

Building heights (pixels, then metres = px ÷ 7.7):

BUILDING	HEIGHT
Main block	58 px ≈ 7.5 m
North wing	46 px ≈ 6 m
South block	40 px ≈ 5.2 m
South-east block	34 px ≈ 4.4 m

These are **estimates for the look**, not measured heights. Say so if asked.

A bug we found and fixed (21 Oct). The North wing didn't rise in 3D. Its height had been written as `h` instead of `z`, so the code read "no height" and the wing stayed flat. Renaming it fixed it.

5. The X-ray lens

This is the feature people remember. Know exactly how it works.

What the user sees

- **At rest:** a small magnifying glass sits in the corner of the plan, with a handwritten "hold & drag me".
- **Pick it up:** press and drag the lens, or (with a mouse) **press and hold anywhere on the plan for 0.3 s**, and the lens flies to your cursor.
- **Drag it:** inside the circle the plan is **magnified 2.2×** and switches to **X-ray mode**:
  - the **real satellite photo** shows through underneath
  - bays get strong colours:

STATUS	X-RAY COLOUR
Free	**yellow**
Booked	**near-black**
Ending	dark with a **yellow outline**
Reserved	white with a **dashed outline**
Yours	yellow with a **thick black outline**

  - every bay gets a tiny label: ID, a word (FREE / BOOKED / ENDING / RSVD / YOURS) and a time
- **The readout card** follows the lens. It shows:
  - the bay under the centre: ID, status, "free till 16:30" / "booked till 15:00" / "frees in 12m" / "driver due 14:30", its type and rate
  - **"In the lens":** a count of free, booked, ending and reserved bays inside the circle

- "Over the road" if the centre isn't on a bay

- **Let go:** the bay under the centre **opens in the booking panel**, and the lens flies back to its corner in 0.75 s.

How it's built

1. Two copies of the map. The normal plan, and a second copy *inside* the lens (`LensClone`, lines 535–537) drawn with `xray` on. X-ray mode adds the photo (`<image href="/kiet-lot.jpg">` at 55% opacity, rotated the same −1°), recolours the bays, hides the cars and adds labels (CSS `islands.css` 716–780).

2. The circle. The lens is a `div` with `border-radius: 50%` and `overflow: hidden`, so you only see the copy through a round window:

- **Laptop:** radius 80 px (160 px across)
- **Phone:** radius 64 px

3. Making it magnify (`place`, lines 603–626). Every time the lens moves to (x, y):

- the lens moves to (x, y)
- the copy inside moves the *opposite* way, so the point under the lens centre lines up with the same point of the real map
- the copy is scaled **2.2×** (`LENS_Z`) with its scale centre (`transform-origin`) at (x, y)

The result looks exactly like a magnifying glass.

4. Smooth at 60 frames a second. While dragging, we **don't** go through React state. We write `style.transform` straight onto the elements. React re-drawing 96 bays on every mouse move would stutter.

5. Working out what's under it (`scan`, lines 639–665):

- `document.elementFromPoint(x, y)` asks the browser "what's at the lens centre?" If it's a bay, we read its ID.
- For the counts, we measure every bay's position and count those within the lens radius ÷ 2.2 (the area of the real map the lens covers), grouped by status.

6. Not too often (`moveTo`, lines 667–689):

- Scanning at most every **70 ms** keeps it cheap.
- A final scan **80 ms after you stop moving** makes sure the card always describes where the lens actually came to rest.
- **That was a real bug we fixed:** before, if you moved fast and stopped, the card could still show the previous bay.

7. Mouse, touch and pen. It uses **pointer events**, one code path for all three.

- **On touch,** the lens sits up and to the left of your finger (offset 0.78 × radius), so your finger doesn't cover what you're looking at.

- **Press-and-hold-anywhere** is mouse-only. On a phone, a long press is for scrolling, so there you grab the lens itself.

8. Accessibility. The lens is decorative (`aria-hidden`). Everything it shows is also in the bay list and the panel, which work with a keyboard and screen readers. The readout card is `aria-live="polite"`, so screen readers announce it.

6. Run it and practise

The site runs at `http://localhost:8787` once Prisha's server is started (`cd backend`, then `python server.py`).

Practise:

1. **Laptop:** drag the lens slowly across row A. Read the card out loud. Let go on a free bay; the panel opens.
2. **Phone:** open the site on a phone on the same Wi-Fi (or the share link). The map is portrait. Grab the lens with your thumb and drag.
3. **3D:** press **3D**, then **Plan**. Name the buildings as they rise: main block, north wing, south block, south-east block.
4. **Row zoom:** press **A**, then **All**.
5. **Dark mode:** the lens and plan follow it.

7. Honest limits

- **Bay counts come from row length ÷ a standard bay width**, not from counting painted lines on the ground. Check row D in person before the event.
- **Building heights are estimates** for the 3D look.
- **The plan is one fixed lot.** Adding another lot means tracing its rows the same way. It's hours of work, but it's manual.
- **The plan itself isn't keyboard-navigable** bay by bay. Keyboard and screen-reader users use the list of all bays instead.
- **The satellite photo is a snapshot.** If the lot is repainted, the trace needs updating.

8. Questions to prepare

The map

Q1. Is this the real lot? Yes. It's traced on a Google Earth capture of the lot beside the KIET football ground. Every bay sits on the photo's own pixels, so the plan lines up with the photo. The lens shows the photo underneath to prove it.

Q2. How did you get real measurements from a photo? From the photo's scale bar: 50 metres = 385 pixels, so 7.7 pixels per metre. A standard bay comes out at 2.5 × 4.9 m, which is a normal car bay.

Q3. Did you draw all 96 bays by hand? We traced each of the five rows by hand: its kerb line, start and end. Bays in a row are the same width, so the code divides each row evenly. That's more accurate than hand-drawing 96 rectangles, and quicker to change.

Q4. How do you know there are 96 bays? Each row's measured length divided by a standard 2.5 m bay. (If you counted row D on the ground, say so.)

Q5. Why is the satellite photo rotated? Satellite captures are rarely perfectly aligned. We turned it 1 degree so the rows run perfectly straight, which keeps the maths simple.

Q6. Why SVG and not an image or a canvas? SVG shapes stay sharp at any zoom, each bay is a real element you can tap, and CSS can colour them for status and dark mode. A canvas would mean redrawing everything by hand on every change.

Q7. Why does the map turn sideways on a laptop? The lot is tall and narrow. On a wide screen we rotate it 90° so it fits. Labels are drawn separately so they stay upright.

Q8. Can you add another lot? Yes, the same way: a satellite capture, its scale bar, mark the rows, enter each row's line, start, end and bay count. Then the site and server can book it.

The lens

Q9. How does the lens work? There's a second copy of the map inside a round window. It's shifted so it lines up with the real map and magnified 2.2×. That copy is drawn in X-ray mode: the satellite photo underneath, strong colours and labels.

Q10. How does it know which bay is under it? The browser has a function, `elementFromPoint`, that tells you what's at a screen position. We call it at the lens centre. For the counts, we check which bays are within the lens radius.

Q11. Why is it smooth? While dragging we move the elements directly instead of asking React to redraw. We also only re-check what's under the lens every 70 ms, plus once more when it stops.

Q12. Does it work on phones? Yes. It uses pointer events, so mouse, touch and pen share one code path. On touch, the lens floats above your finger so you can see it.

Q13. What's the point of the lens, not just colours? It's faster to scan an area. You can sweep across a row and see at a glance how many bays are free, what's leaving soon and who's on the way, with the real photo underneath. Then let go to book.

Q14. Any bugs you fixed?

- The lens card could show the previous bay if you moved fast and stopped. We added a final check 80 ms after the lens stops.
- In 3D, the North wing didn't rise because its height was stored under the wrong name. Both are fixed.

3D

Q15. Did you use a 3D library? No, it's CSS 3D. We tilt the plan with `rotateX` and `rotateZ` under a perspective, then build each building from flat walls stood up at the right angles, with a roof lifted by its height.

Q16. How do the buildings grow? Their height is multiplied by a CSS variable `--lift` that goes from 0 to 1. We register it with `@property` so the browser can animate it. It starts 0.35 s after the tilt.

Q17. Are the building heights real? They're estimates for the look, about 4 to 7.5 metres. The ground plan is measured; the heights aren't.

Q18. Why 3D at all? Recognition. Flat plans are hard to read if you don't know the place. With the main block and wings standing up, you instantly see where you are.

General

Q19. What did you find hardest? Making the plan match the real lot exactly, then making the lens line up perfectly with it at 2.2× zoom in both orientations.

Q20. Did you use AI? Answer truthfully, the same way as the rest of the team. If you did, say so plainly and show you understand the result. For example: "Yes, we used AI tools while building. We gave it our marked-up photo and our measurements, checked the result against the lot, and I can explain how every part works."

---

9. Words you should know

WORD	MEANING
Satellite capture	A screenshot of the lot from Google Earth.
Scale bar	The ruler on a map that says how many metres a length is.

WORD	MEANING
Pixel coordinates	Positions measured in pixels of the photo, (x across, y down).
Pitch	The distance from one bay to the next along a row.
SVG	A picture made of shapes in code; sharp at any size.
viewBox	Which part of the SVG's coordinates is shown in its box.
Transform	Moving, turning or scaling an element (`translate` , `rotate` , `scale`).
transform-origin	The point a scale or rotation happens around.
Perspective	Makes far things look smaller, like a real camera.
rotateX / rotateZ	Tip forward and back / turn flat on the table.
translateZ	Move towards or away from the viewer (up, for a roof).
clip-path	Cuts an element to a shape, here the roof outline.
@property	Registers a CSS variable's type so it can be animated.
Pointer events	One kind of browser event for mouse, touch and pen.
elementFromPoint	Browser function: "what element is at this screen position?"
Throttle	Doing something at most every N ms, however often it's asked.
IntersectionObserver	Tells you when an element scrolls into view.
ResizeObserver	Tells you when an element changes size.

10. Numbers to remember

Photo	Google Earth · **784 × 1601 px** · turned **−1°**
Scale	50 m = 385 px → **7.7 px per metre**
Bays	**96** in **5 rows**: A 33 · B 10 · C 24 · D 6 · E 23
Standard bay	**2.5 × 4.9 m** (19.4 × 38 px)
Special bays	EV: C-01 to C-04 · step-free: D-01, D-02, A-33 · row B compact
Orientation	portrait up to **980 px** wide · landscape above (turned −90°)
Lens	radius **80 px** laptop / **64 px** phone · zoom **2.2×** · scan every **70 ms** + final scan after **80 ms**
Lens pick-up	drag it, or hold on the plan **0.3 s** (mouse) · flies back in **0.75 s**
3D	perspective **2400 px** · tilt **54°**, turn **−32°** · tilt 1.2 s, buildings rise after 0.35 s
Building heights	main 7.5 m · north wing 6 m · south 5.2 m · south-east 4.4 m (estimates)
Row zoom	up to **3.4×**, 1.1 s